



Compositional Verification of Smart Contracts Through Communication Abstraction

Scott Wesley



Jorge Navas



Valentin Wüstholtz



Maria Christakis



Arie Gurfinkel
Richard Trefler



Motivation and Background: The Problem of Smart Contract Verification

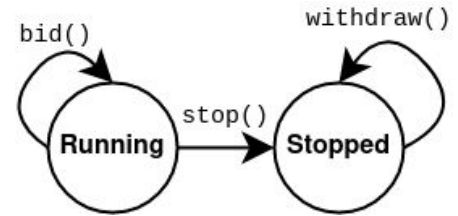


Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.

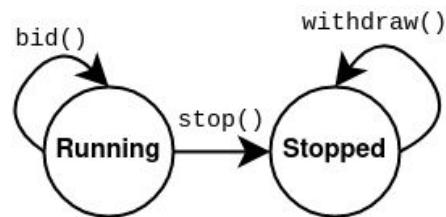
Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as open bid auctions.



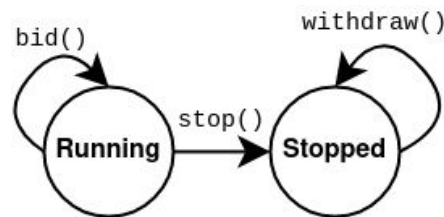
Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as open bid auctions.
- **Solidity** smart contracts facilitate **information flow** (e.g., *placing bids*) between many users with **unique addresses**.



Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as open bid auctions.
- **Solidity** smart contracts facilitate **information flow** (e.g., *placing bids*) between many users with **unique addresses**.
- For example, the **program given to the right** is the interface for a Solidity smart contract that implements an **open bid auction with a designated manager**.



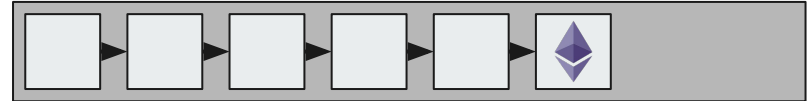
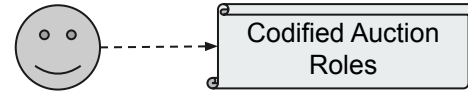
```
1. contract Auction {
2.     constructor(address manager) public { ... }
3.     function bid() public payable { ... }
4.     function withdraw() public { /... }
5.     function stop() public { ... }
6. }
```



Motivation: Trust via Source Code Verification

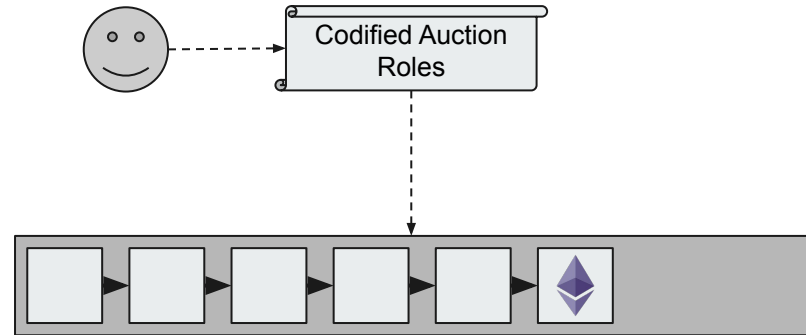
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., *an auction*) as a Solidity smart contract.



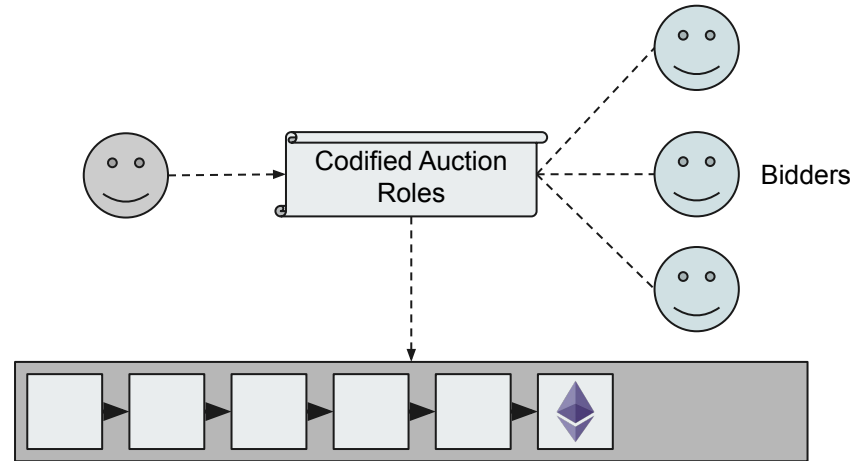
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., *an auction*) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.



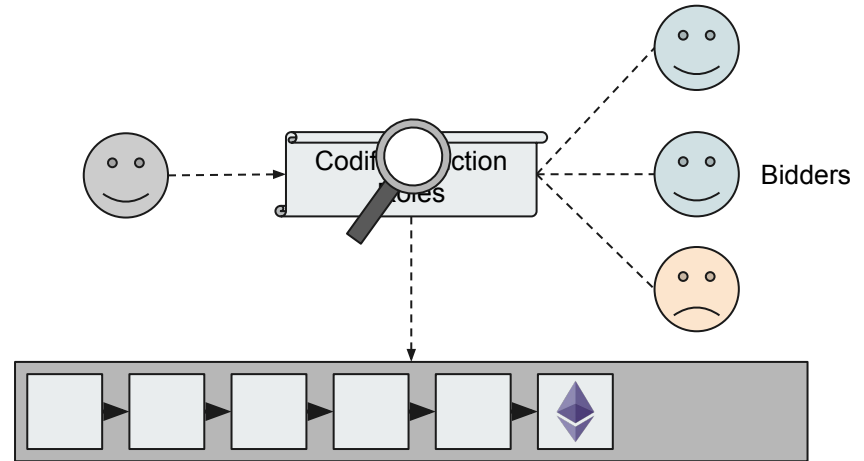
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., an auction) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.
3. Untrusting parties interact via the contract as a mechanism to enforce cooperation.



Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., an auction) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.
3. Untrusting parties interact via the contract as a mechanism to enforce cooperation.
4. To ensure that trust can be placed in the smart contract, each party **should want the source code of the smart contract verified.**





Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```

Notation: States and Traces

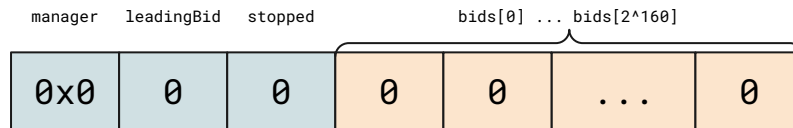
```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```

manager leadingBid stopped

0x0	0	0
-----	---	---

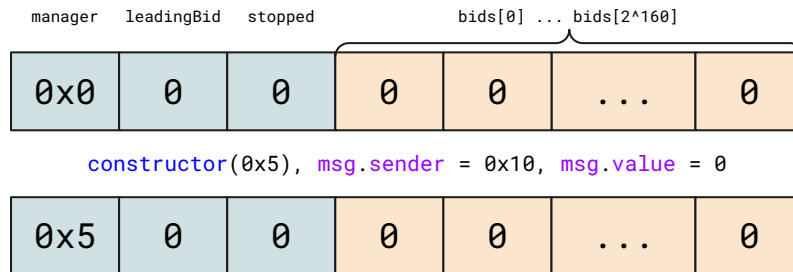
Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```



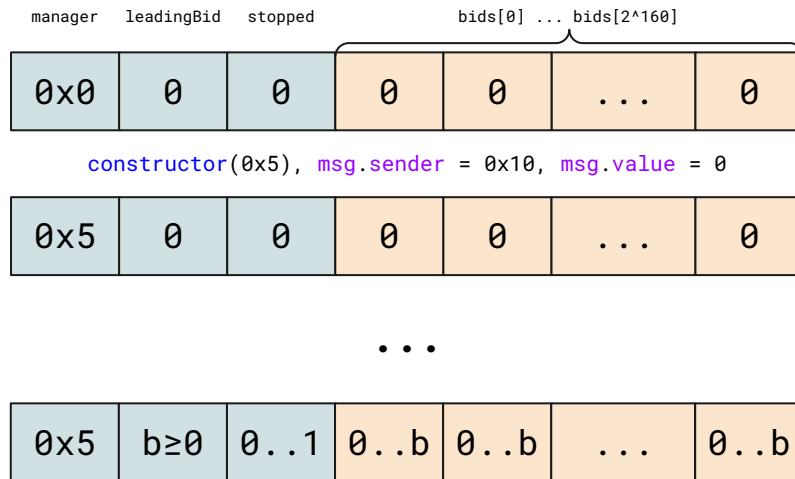
Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```



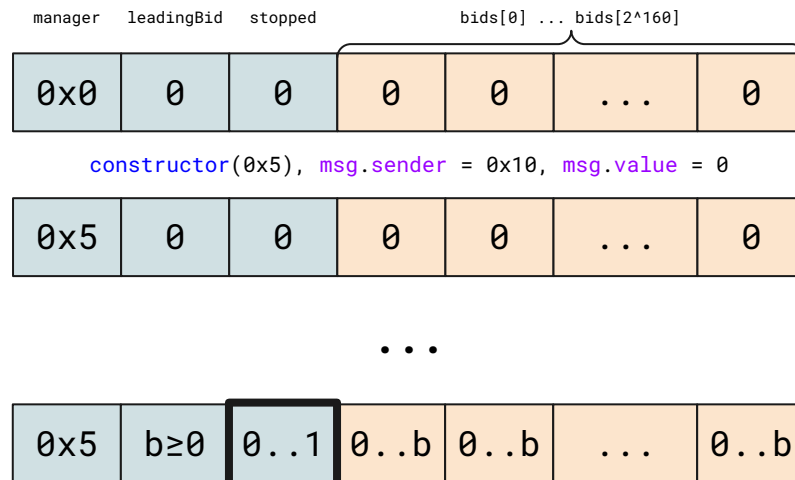
Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.
06.   function bid() public payable {
07.     // Checks permission to place bid.
08.     require(!stopped, "Auction must be ongoing.");
09.     require(msg.sender != manager, "Bidder must not be manager.");
10.     // Checks bid is valid.
11.     uint curBid = bids[msg.sender];
12.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
13.     require(curBid + msg.value > leadingBid, "Bid too small.");
14.     // Records bid.
15.     bids[msg.sender] = curBid + msg.value;
16.     leadingBid = bids[msg.sender];
17.   }
18.
19.   constructor(address manager) public { /* ... */ }
20.   function withdraw() public { /* ... */ }
21.   function stop() public { /* ... */ }
22. }
```



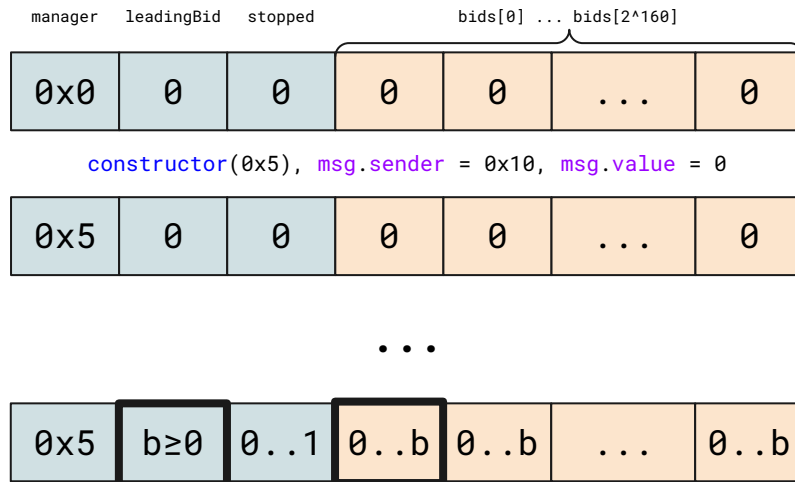
Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```



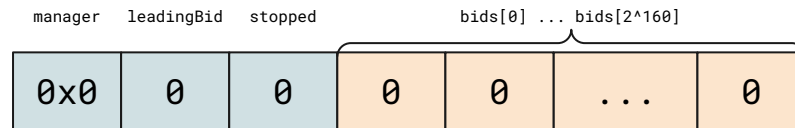
Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
22. }
```



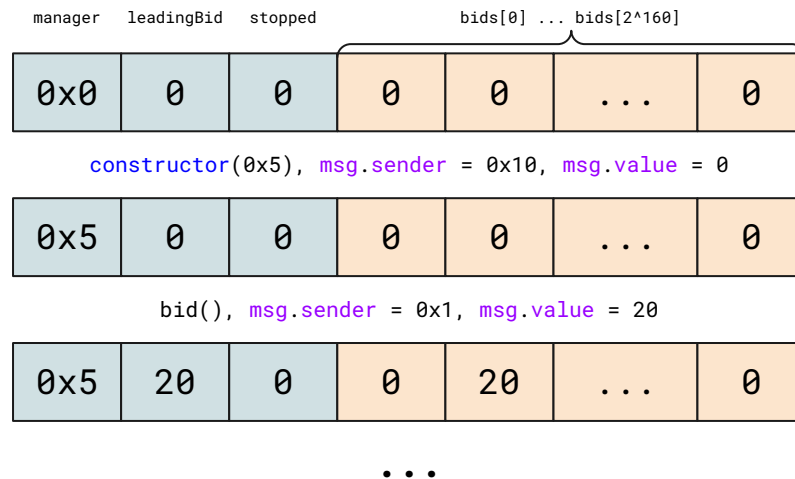
Challenge: Intractable Verification Problems

1. The state space is intractably large.
 - For example, the bids mapping in Auction stores 2^{160} values each of size 2^{256} bits.



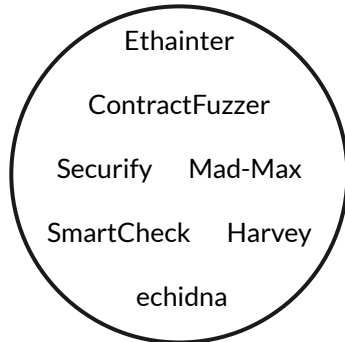
Challenge: Intractable Verification Problems

1. The state space is **intractably large**.
 - For example, the bids mapping in Auction stores 2^{160} values each of size 2^{256} bits.
2. Sequences of transactions are **unbounded or intractably long**.
 - For example, Auction permits traces of up to $2^{161} + 1$ transactions long.



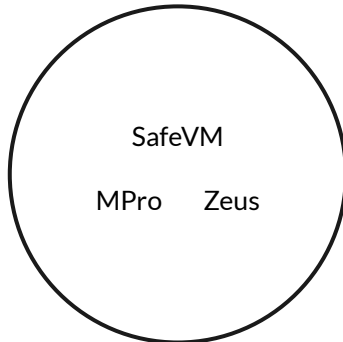
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



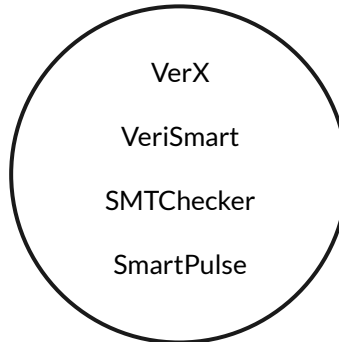
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



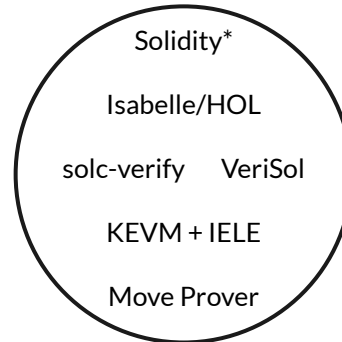
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



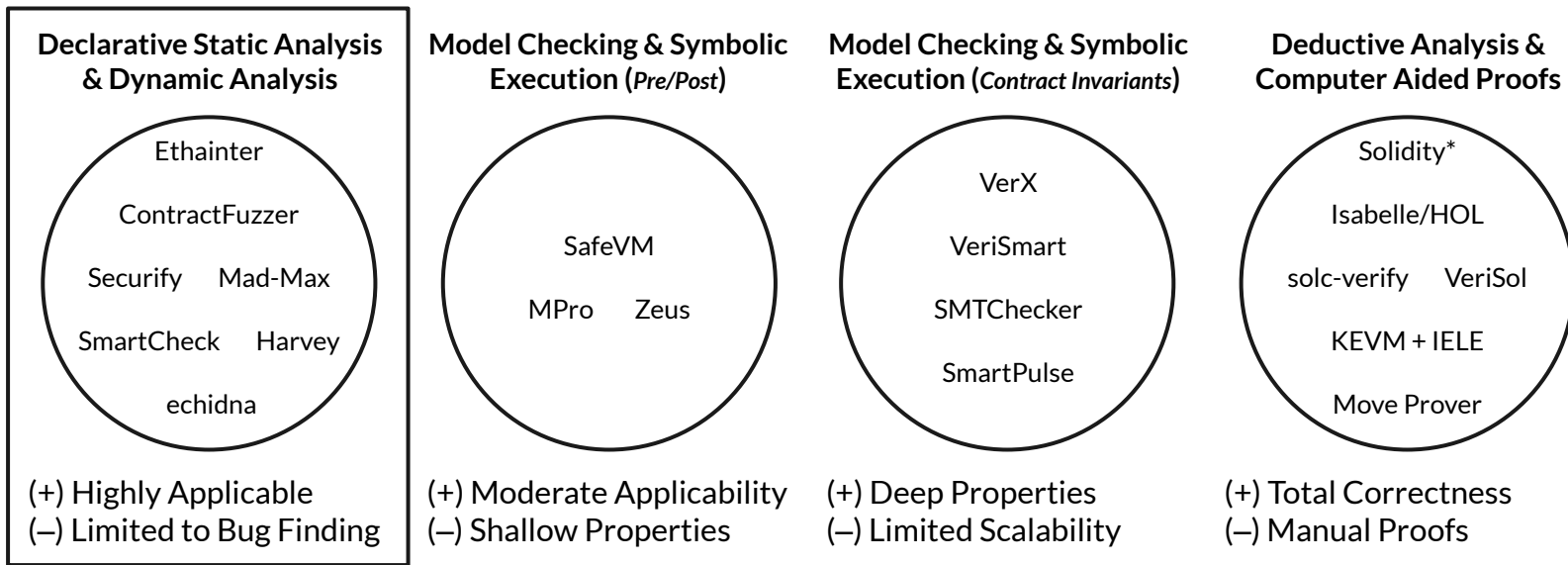
(+) Deep Properties
(-) Limited Scalability

Deductive Analysis & Computer Aided Proofs



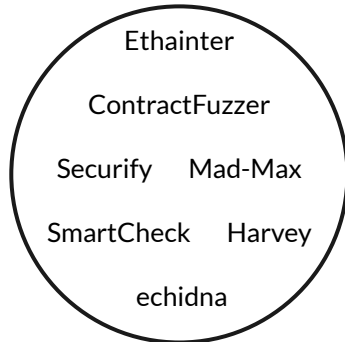
(+) Total Correctness
(-) Manual Proofs

Challenges: Current Solutions



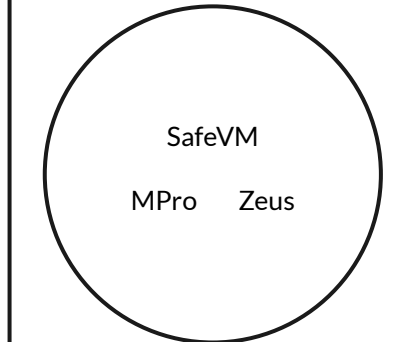
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



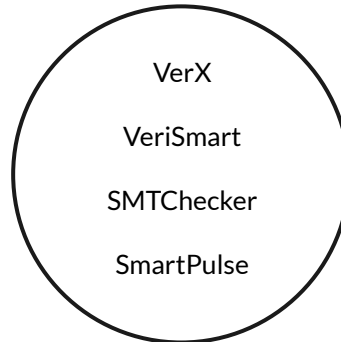
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



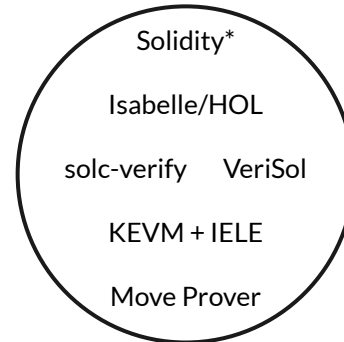
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



(+) Deep Properties
(-) Limited Scalability

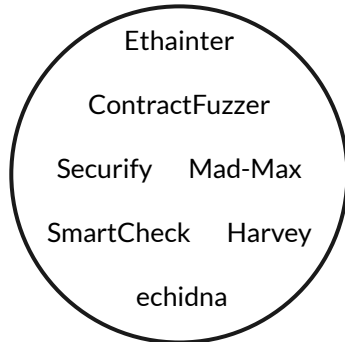
Deductive Analysis & Computer Aided Proofs



(+) Total Correctness
(-) Manual Proofs

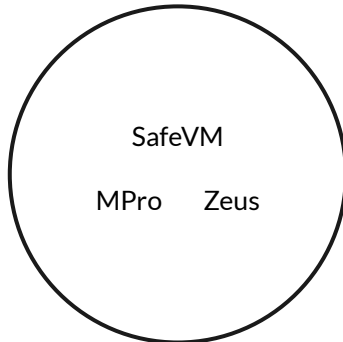
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



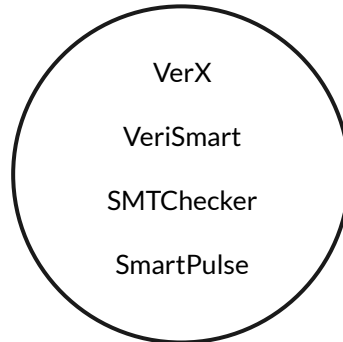
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



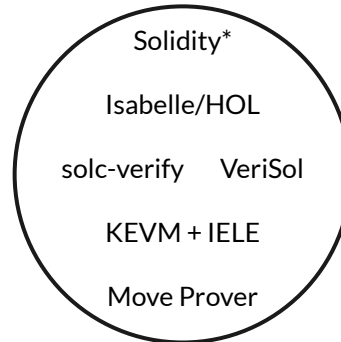
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



(+) Deep Properties
(-) Limited Scalability

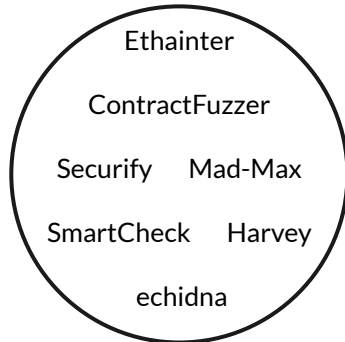
Deductive Analysis & Computer Aided Proofs



(+) Total Correctness
(-) Manual Proofs

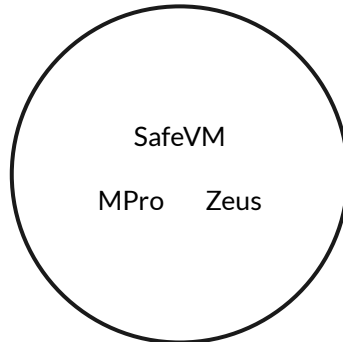
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



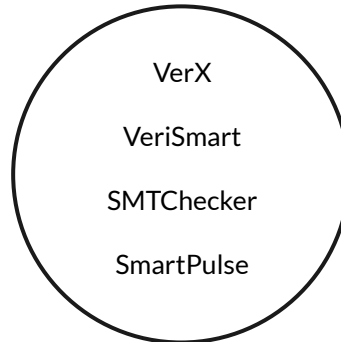
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



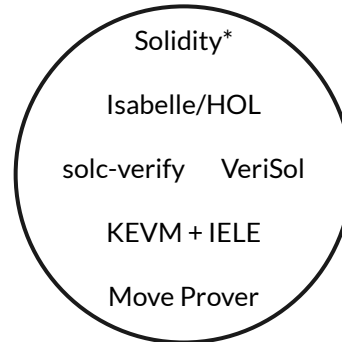
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



(+) Deep Properties
(-) Limited Scalability

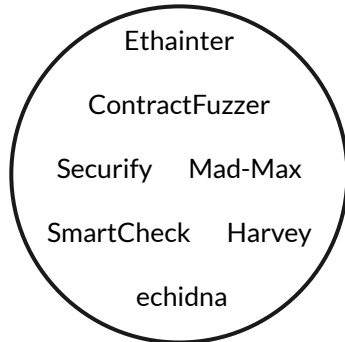
Deductive Analysis & Computer Aided Proofs



(+) Total Correctness
(-) Manual Proofs

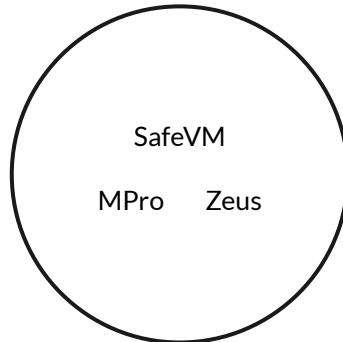
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



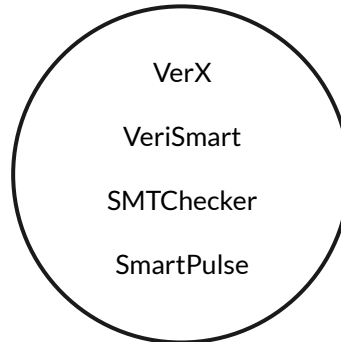
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



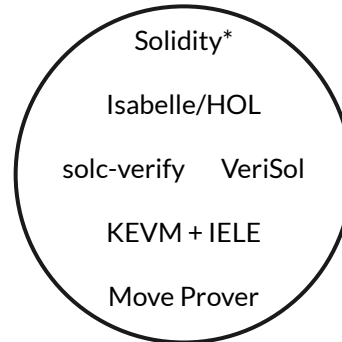
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



(+) Deep Properties
(-) Limited Scalability

Deductive Analysis & Computer Aided Proofs



(+) Total Correctness
(-) Manual Proofs

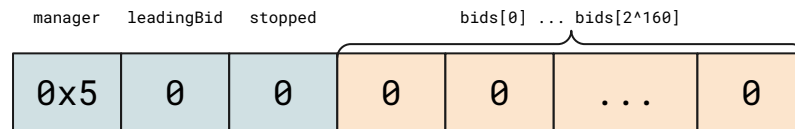
Our Approach:

Local Bundle Abstractions



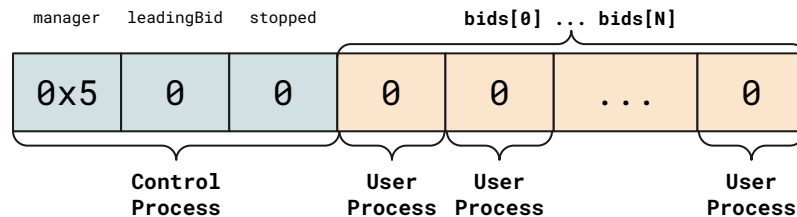
Approach: Our Insight

- It is often easier to verify a parameterized system for any number of users, than a large instance of the system. From this insight:



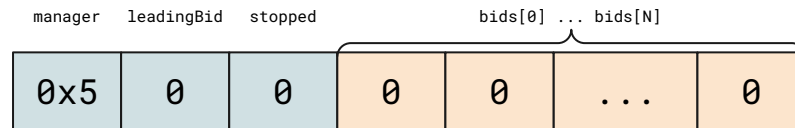
Approach: Our Insight

- It is often easier to verify a parameterized system for any number of users, than a large instance of the system. From this insight:
 - a. Generalize Solidity smart contracts to parameterized networks with control and user processes.



Approach: Our Insight

- It is often easier to verify a parameterized system for any number of users, than a large instance of the system. From this insight:
 - a. Generalize Solidity smart contracts to parameterized networks with control and user processes.
 - b. Characterize communication patterns between user processes.





Approach: Our Insight

- It is often easier to verify a parameterized system for any number of users, than a large instance of the system. From this insight:
 - a. Generalize Solidity smart contracts to parameterized networks with control and user processes.
 - b. Characterize communication patterns between user processes.
 - c. Identify groups of users that communicate, and characterize users that are interchangeable between groups.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x5	0	0	0	0	0	0

Approach: Our Insight

- It is often easier to verify a parameterized system for any number of users, than a large instance of the system. From this insight:
 - a. Generalize Solidity smart contracts to parameterized networks with control and user processes.
 - b. Characterize communication patterns between user processes.
 - c. Identify groups of users that communicate, and characterize users that are interchangeable between groups.
 - d. Use representative users to simplify analysis.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x5	0	0	0	0	0	0
bid(), msg.sender = 0x1, msg.value = 10						
0x5	10	0	0	0	0	0-10



Approach: A short example

This example shows that Auction satisfies the property: “The bid of each user is at most the leading bid.” This property is also used as an interference invariant.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x0	0	0	0	0	0	0

Approach: A short example

This example shows that Auction satisfies the property: “The bid of each user is at most the leading bid.” This property is also used as an interference invariant.

An unbounded sequence is executed following contract construction.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x0	0	0	0	0	0	0
constructor(0x2), msg.sender = 0x2, msg.value = 0						
0x2	0	0	0	0	0	0
bid(), msg.sender = 0x1, msg.value = 20						
0x2	20	0	0	0	0	20

Approach: A short example

This example shows that Auction satisfies the property: “The bid of each user is at most the leading bid.” This property is also used as an interference invariant.

An unbounded sequence is executed following contract construction.

After each transaction, the interference invariant is checked. In this example, the interference invariant is satisfied, as the leading bid is 20.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x0	0	0	0	0	0	0
constructor(0x2), msg.sender = 0x2, msg.value = 0						
0x2	0	0	0	0	0	0
bid(), msg.sender = 0x1, msg.value = 20						
0x2	20	0	0	0	0	20

Approach: A short example

This example shows that Auction satisfies the property: “The bid of each user is at most the leading bid.” This property is **also used as an interference invariant**.

An unbounded sequence is executed following contract construction.

After each transaction, the interference invariant is checked. In this example, the interference invariant is satisfied, as the leading bid is 20.

The interference invariant is used to abstract the state of each user.

manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x0	0	0	0	0	0	0
constructor(0x2), msg.sender = 0x2, msg.value = 0						
0x2	0	0	0	0	0	0
bid(), msg.sender = 0x1, msg.value = 20						
0x2	20	0	0..20	0..20	0..20	0..20

Approach: A short example

This example shows that Auction satisfies the property: “The bid of each user is at most the leading bid.” This property is **also used as an interference invariant**.

An unbounded sequence is executed following contract construction.

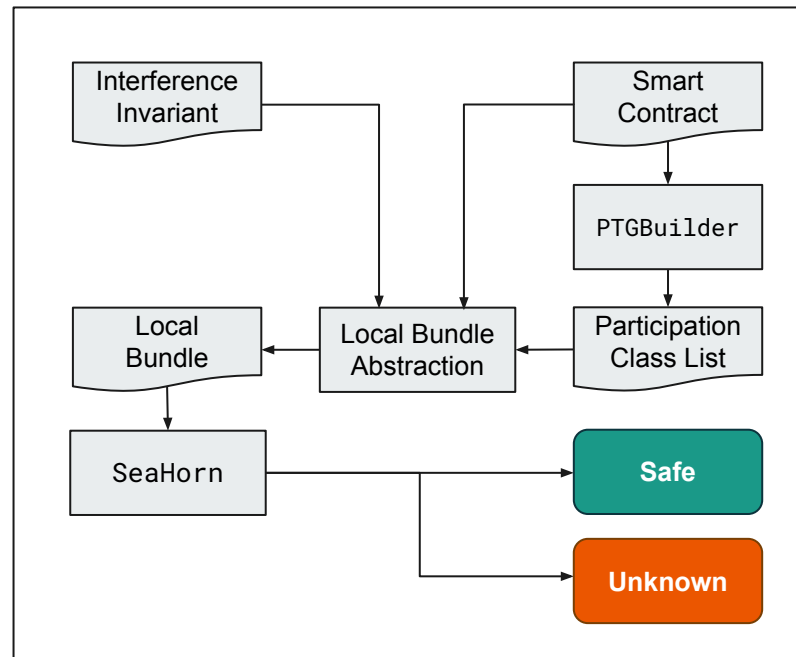
After each transaction, the interference invariant is checked. In this example, the interference invariant is satisfied, as the leading bid is 20.

The interference invariant is used to abstract the state of each user.

The over-approximation of reachable states shows that all bids are less than the leading bid.

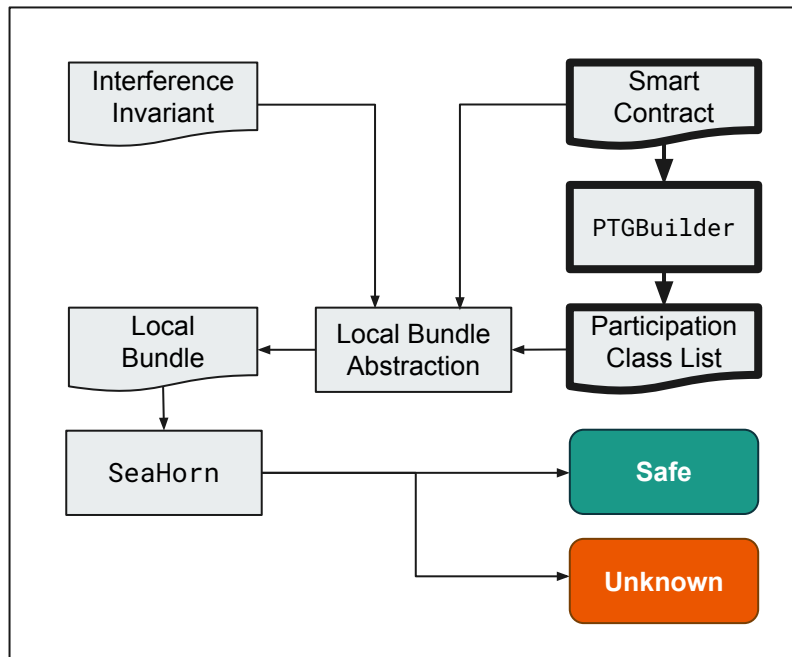
manager	leadingBid	stopped	Bid of zero-account	Bid of Auction	Bid of manager	Bid of msg.sender
0x0	0	0	0	0	0	0
constructor(0x2), msg.sender = 0x2, msg.value = 0						
0x2	0	0	0	0	0	0
bid(), msg.sender = 0x1, msg.value = 20						
0x2	20	0	0..20	0..20	0..20	0..20
...						
0x2	$b \geq 0$	0-1	0..b	0..b	0..b	0..b

Approach: Technical Details



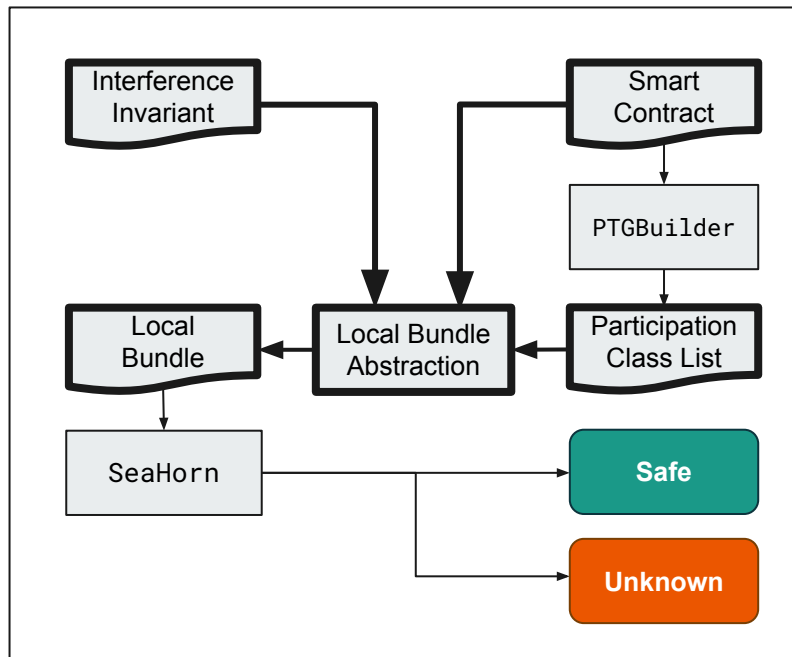
Approach: Technical Details

- To identify participation classes (i.e., groups of similar users in a transaction) an algorithm called **PTGBuilder** applies **dataflow analysis** to identify uses that influence the outcomes of transactions.



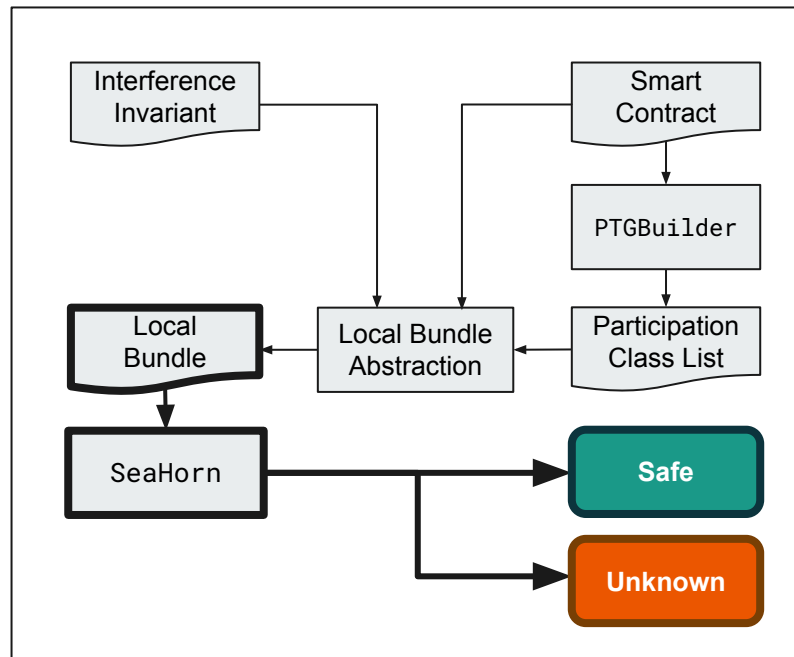
Approach: Technical Details

- To **identify participation classes** (i.e., groups of similar users in a transaction) an algorithm called **PTGBuilder** applies **dataflow analysis** to identify uses that influence the outcomes of transactions.
- **Local bundle abstractions** are introduced to abstract smart contracts by **representative users** with *local* communication. (The preceding example used local bundle abstraction to reduce from 2^{160} users to 4 users.)



Approach: Technical Details

- To **identify participation classes** (i.e., groups of similar users in a transaction) an algorithm called **PTGBuilder** applies **dataflow analysis** to identify users that influence the outcomes of transactions.
- **Local bundle abstractions** are introduced to abstract smart contracts by **representative users** with *local* communication. (The preceding example used local bundle abstraction to reduce from 2^{160} users to 4 users.)
- **Software model checking** (e.g., *SeaHorn*) is used to **verify** that the local bundle abstraction satisfies a **provided property**.





Key Results of the VerX Case Study

Contracts			SMARTACE			VERX
Name	Prop.	LOC	Time Inv.	Size	Users	Time
Alchemist	3	401	7	0	7	29
ERC20	9	599	12	1	5	158
Melon	16	462	30	0	7	408
MRV	5	868	2	0	7	887
Overview	4	66	4	0	8	211
PolicyPal	4	815	26	0	8	20,773
Zebi	5	1,209	8	0	7	77
Zilliqa	5	377	8	0	7	94
Brickblock	6	549	13	0	10	191
Crowdsale	9	1,198	223	0	8	261
ICO	8	650	371	0	16	6,817
VUToken	5	1,120	19	0	10	715
Mana	4	885	—	—	—	41,409



Key Results of the VerX Case Study

Conformance: 8 out of 13 VerX contracts were supported by SmartACE. **After manual abstraction, 12 out of 13 VerX benchmarks were supported.**

Contracts			SMARTACE			VERX
Name	Prop.	LOC	Time	Inv.	Size Users	Time
Alchemist	3	401	7	0	7	29
ERC20	9	599	12	1	5	158
Melon	16	462	30	0	7	408
MRV	5	868	2	0	7	887
Overview	4	66	4	0	8	211
PolicyPal	4	815	26	0	8	20,773
Zebi	5	1,209	8	0	7	77
Zilliqa	5	377	8	0	7	94
Brickblock	6	549	13	0	10	191
Crowdsale	9	1,198	223	0	8	261
ICO	8	650	371	0	16	6,817
VUToken	5	1,120	19	0	10	715
Mana	4	885	—	—	—	41,409



Key Results of the VerX Case Study

Conformance: 8 out of 13 VerX contracts were supported by SmartACE. **After manual abstraction, 12 out of 13 VerX benchmarks were supported.**

Verification: All conforming contracts were verified by SmartACE.

Contracts			SMARTACE			VERX
Name	Prop.	LOC	Time	Inv.	Size Users	Time
Alchemist	3	401	7	0	7	29
ERC20	9	599	12	1	5	158
Melon	16	462	30	0	7	408
MRV	5	868	2	0	7	887
Overview	4	66	4	0	8	211
PolicyPal	4	815	26	0	8	20,773
Zebi	5	1,209	8	0	7	77
Zilliqa	5	377	8	0	7	94
Brickblock	6	549	13	0	10	191
Crowdsale	9	1,198	223	0	8	261
ICO	8	650	371	0	16	6,817
VUToken	5	1,120	19	0	10	715
Mana	4	885	—	—	—	41,409

Key Results of the VerX Case Study

Conformance: 8 out of 13 VerX contracts were supported by SmartACE. **After manual abstraction, 12 out of 13 VerX benchmarks were supported.**

Verification: All conforming contracts were verified by SmartACE.

Performance: In all cases, SmartACE outperformed VerX. **In 10 out of 12 cases, SmartACE achieved a speedup of at least 10x.**

Contracts			SMARTACE			VERX
Name	Prop.	LOC	Time	Inv.	Size Users	Time
Alchemist	3	401	7	0	7	29
ERC20	9	599	12	1	5	158
Melon	16	462	30	0	7	408
MRV	5	868	2	0	7	887
Overview	4	66	4	0	8	211
PolicyPal	4	815	26	0	8	20,773
Zebi	5	1,209	8	0	7	77
Zilliqa	5	377	8	0	7	94
Brickblock	6	549	13	0	10	191
Crowdsale	9	1,198	223	0	8	261
ICO	8	650	371	0	16	6,817
VUToken	5	1,120	19	0	10	715
Mana	4	885	—	—	—	41,409

Conclusions

Smart contracts are **parameterized** systems with many **users** and a single **controller**.

Parameterized verification is **necessary** to ameliorate the state explosion induced by users.

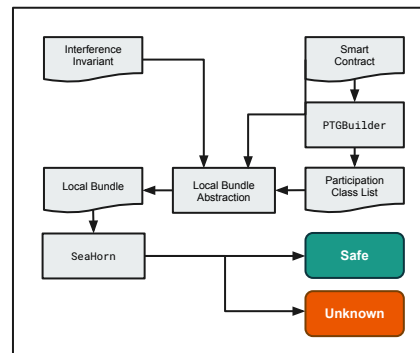
We introduce local bundle abstraction (LBA) that captures the behavior of many users by a few:

- LBA is automatically constructed from source code of a contract using static analysis;
- LBA is a sequential program that can be analyzed by of-the-shelf software model checkers.

New Tool: SmartACE -- based on Solidity compiler and SeaHorn model checker:

- Order of magnitude faster than VerX, while verifying for arbitrary many users!

See paper for additional case studies.



SmartACE and Experiments are available on GitHub: <https://github.com/contract-ace>